Trabajo Práctico Integrador

Backend de Aplicaciones — 2026

> **Versión preliminar — Lineamientos generales**
> Para empezar a pensar

Introducción

El Trabajo Práctico Integrador tiene como objetivo diseñar, implementar, integrar, asegurar y desplegar una **solución backend basada en microservicios**, aplicando de manera integrada los conceptos desarrollados durante la asignatura.

A diferencia de actividades anteriores, el TPI **no propone un dominio completamente especificado ni determina de antemano la solución funcional o arquitectónica**.

Cada equipo deberá seleccionar un dominio de problema, comprenderlo, establecer su alcance y diseñar una solución que permita demostrar los conocimientos adquiridos durante el cursado.

Como parte de este documento se presenta un conjunto de **dominios de problema sugeridos**. Estos dominios funcionan únicamente como disparadores y podrán ser extendidos, reinterpretados o reemplazados por una propuesta propia del equipo.

El problema finalmente seleccionado deberá permitir construir una solución con suficiente riqueza funcional como para justificar las decisiones de diseño, integración, seguridad y despliegue adoptadas.

Objetivo del trabajo

Esperamos que cada equipo construya una solución backend en la que sea posible identificar y justificar:

- responsabilidades funcionales claramente diferenciadas;
- reglas de negocio relevantes;
- colaboración entre componentes;
- integración con información proveniente de sistemas externos;
- autenticación y autorización;
- reutilización de componentes;
- exposición controlada de servicios;
- persistencia de información;
- documentación de las APIs;
- contenerización y despliegue integral de la solución.

El objetivo no es solamente lograr que un conjunto de servicios funcione.

Esperamos que el equipo pueda **explicar por qué la solución fue diseñada de esa manera, qué alternativas fueron consideradas y qué consecuencias tienen las decisiones tomadas**.

Definición del problema

Cada equipo deberá comenzar por **comprender y definir el problema que pretende resolver**.

Podrá utilizar uno de los dominios sugeridos como punto de partida, extenderlo, reinterpretarlo o proponer un dominio diferente.

Como parte de este proceso, el equipo debería lograr identificar, como mínimo:

- contexto general del problema;
- actores involucrados;
- principales procesos;
- información relevante;
- reglas de negocio identificadas;
- responsabilidades que deberá asumir el backend;
- información que deberá obtenerse desde sistemas externos.

> **Esta definición no constituye un entregable documental en sí misma.**
>
> El objetivo es que el equipo realice el análisis necesario para construir una **visión compartida del problema**, acordar qué pretende resolver y disponer de una base común sobre la cual diseñar y desarrollar la solución.

No se espera que esta definición permanezca inalterable durante todo el trabajo.

El equipo podrá **iterar sobre el problema y sobre el diseño de la solución** a medida que descubra nuevas necesidades, restricciones o alternativas.

Estas iteraciones forman parte del trabajo esperado. Las decisiones relevantes y los cambios significativos deberán poder ser explicados durante la presentación y defensa del TPI.

Diseño de la solución

Antes de comenzar la implementación, el equipo deberá elaborar:

- un **Diagrama de Contexto — C1** utilizando C4 Model;
- un **Diagrama de Contenedores — C2** utilizando C4 Model;

siguiendo la propuesta de Simon Brown.

Los diagramas deberán permitir comprender progresivamente la solución y deberán identificar, según el nivel correspondiente:

- actores;
- sistema a desarrollar;
- sistemas externos;
- API Gateway;
- microservicios funcionales;
- Identity Provider;
- DBMS;
- integraciones externas;
- componentes de infraestructura relevantes;
- relaciones y comunicaciones entre los diferentes componentes.

El diseño inicial **no constituye un contrato inalterable**.

Esperamos que evolucione a medida que el equipo profundice el problema, implemente la solución y descubra nuevas restricciones o alternativas.

La versión presentada en la entrega deberá representar la **arquitectura efectivamente implementada**.

Diagramas de referencia

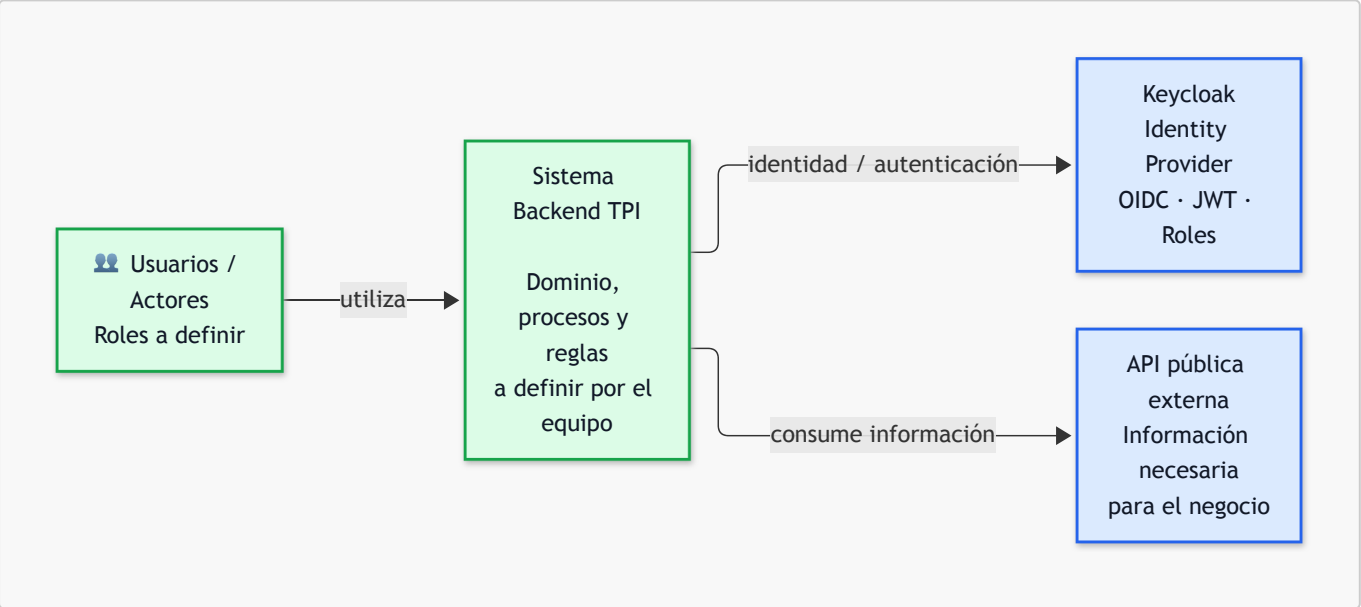
Los siguientes diagramas tienen como único objetivo representar las **restricciones establecidas por el TPI y los espacios de decisión que corresponden a cada equipo**.

No constituyen una arquitectura de referencia que deba reproducirse literalmente.

En ambos diagramas:

- 🟦 **Restricción del TPI:** componente, integración o capacidad cuya existencia está establecida por el enunciado.
- 🟩 **Decisión del equipo:** responsabilidad, contenido o diseño que deberá surgir del análisis realizado por el grupo.

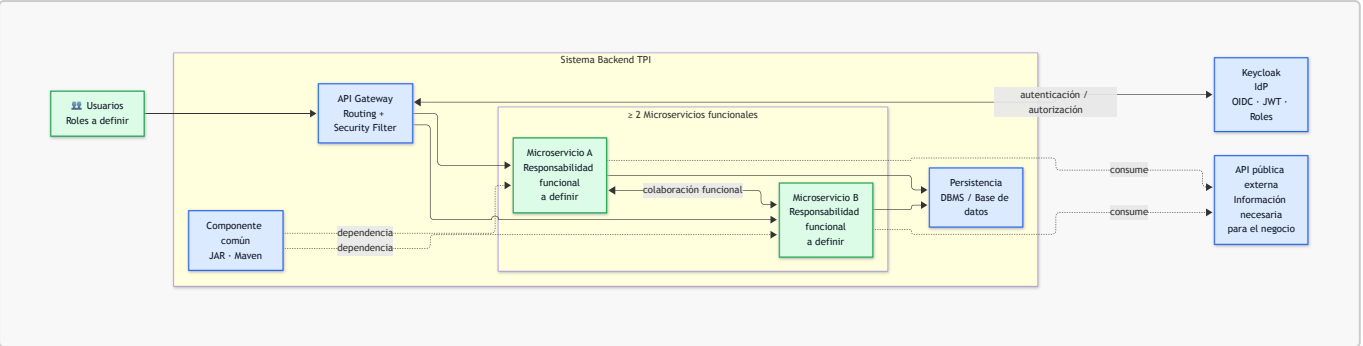
C1 — Diagrama de Contexto genérico



En este nivel todavía **no interesa mostrar cómo está construido internamente el backend**.

El C1 deberá permitir comprender qué problema resuelve el sistema, quiénes interactúan con él y qué otros sistemas externos participan de su contexto.

C2 — Diagrama de Contenedores genérico



El diagrama representa que la existencia de **al menos dos microservicios funcionales** constituye una restricción del TPI, pero **sus responsabilidades, límites e interacciones deberán ser descubiertos y diseñados por cada equipo**.

De la misma manera, se representan únicamente las relaciones necesarias para comprender las restricciones generales. Cada solución podrá incorporar otros componentes, relaciones e integraciones cuando el diseño adoptado lo requiera.

Arquitectura de la solución

La solución deberá implementarse siguiendo una **arquitectura basada en microservicios**, incorporando un API Gateway como punto de entrada al backend.

Microservicios funcionales

La solución deberá contener **al menos dos microservicios funcionales desarrollados en Java con Spring Boot**.

La división entre microservicios deberá responder a **responsabilidades funcionales coherentes dentro del dominio elegido**.

No será suficiente dividir arbitrariamente una aplicación en dos proyectos ni implementar dos conjuntos independientes de operaciones CRUD únicamente para satisfacer este requisito.

Cada microservicio deberá poseer una responsabilidad identificable dentro del dominio y mantener la información necesaria para cumplirla.

Cada equipo deberá descubrir y justificar los límites funcionales apropiados para su propio dominio.

Colaboración entre microservicios

La solución deberá contener procesos que requieran **colaboración real entre microservicios**.

Al menos un proceso significativo de negocio deberá involucrar la participación de más de un microservicio funcional.

La interacción deberá responder a una necesidad real del dominio y no haber sido incorporada artificialmente con el único propósito de demostrar comunicación entre servicios.

Por ejemplo, si un microservicio administra clientes y otro administra pedidos, el servicio responsable de los pedidos podría necesitar obtener información del cliente para ejecutar determinadas operaciones de negocio.

El ejemplo anterior es únicamente ilustrativo y **no establece una división esperada para los trabajos**.

Se evaluará especialmente:

- qué servicio inicia la interacción;
- qué información necesita de otros componentes;
- qué responsabilidad conserva cada servicio;
- cómo se resuelven situaciones en las que otro componente no responde;
- qué información pertenece a cada responsabilidad funcional;
- qué decisiones permiten mantener un nivel razonable de independencia entre los servicios.

No se prescribe una única estrategia para resolver estas interacciones.

API Gateway

La solución deberá disponer de un **API Gateway como punto de entrada a los servicios backend**.

El Gateway deberá encargarse del enrutamiento hacia los diferentes microservicios y participar de la estrategia de seguridad definida por el equipo.

Los consumidores externos no deberían necesitar conocer la topología interna de la solución para utilizar sus funcionalidades.

La implementación deberá incorporar además un **Security Filter en el Gateway**, acorde con el esquema de autenticación y autorización adoptado.

La existencia del Gateway es una restricción arquitectónica del TPI. Su configuración, las rutas expuestas y su participación concreta dentro de la estrategia de seguridad forman parte de las decisiones que deberá tomar y justificar cada equipo.

Integración con una API pública externa

La solución deberá consumir **al menos una API pública externa accesible a través de Internet**.

La integración deberá proporcionar información que resulte **genuinamente necesaria para resolver alguna funcionalidad o regla de negocio** de la solución.

No se considerará una integración significativa aquella que únicamente incorpore información decorativa, datos accesorios o valores que razonablemente podrían almacenarse como configuración propia del sistema.

Por ejemplo, utilizar una API exclusivamente para consultar el valor de cambio entre monedas **no satisface por sí mismo este requerimiento**.

Esperamos que el equipo pueda explicar:

- por qué necesita obtener esa información externamente;
- qué proceso de negocio depende de ella;
- cómo se realiza la integración;
- qué información se utiliza;
- qué sucede cuando el servicio externo no se encuentra disponible o devuelve una respuesta inesperada.

Se exige **al menos una integración externa**, pero no se establece una cantidad máxima.

La cantidad de integraciones incorporadas no será considerada por sí misma una medida de complejidad. Se evaluará fundamentalmente **qué aportan al problema y cómo participan de la solución**.

Identidad, autenticación y autorización

La solución deberá utilizar **Keycloak como Identity Provider (IdP)**.

La arquitectura de seguridad deberá incorporar:

- **OpenID Connect (OIDC)**;
- tokens **JWT**;
- **Spring Security**;
- roles administrados mediante Keycloak;
- autenticación de los usuarios;
- autorización de las operaciones de acuerdo con los roles definidos;
- integración de la seguridad con el API Gateway.

El equipo deberá identificar los diferentes tipos de usuarios que interactúan con su solución y determinar qué operaciones puede realizar cada uno.

No se prescribe una única estrategia para distribuir las responsabilidades de seguridad entre Gateway y microservicios.

Cada equipo deberá diseñar y justificar cómo establece su **trusted zone**, qué componentes pueden comunicarse entre sí y qué mecanismos utiliza para asegurar que las operaciones protegidas sólo puedan ser ejecutadas por usuarios debidamente autenticados y autorizados.

Componente común reutilizable

La solución deberá incluir al menos un **componente Java reutilizable empaquetado como JAR**.

Este componente deberá construirse como un **artefacto Maven independiente** y deberá ser utilizado como dependencia por, al menos, los dos microservicios funcionales principales.

Como mecanismo mínimo, el artefacto deberá ser **desplegado/instalado en el repositorio Maven local (`.m2`)** y posteriormente referenciado desde los microservicios mediante las dependencias correspondientes de sus archivos `pom.xml`.

El equipo podrá optar por utilizar un **repositorio Maven compartido** u otro mecanismo de distribución de artefactos si desea explorar una solución más completa. Esto no constituye un requisito adicional: el repositorio Maven local representa el mecanismo mínimo esperado.

No se espera que el código fuente del componente sea simplemente copiado entre proyectos. Los microservicios deberán **consumir el artefacto generado como una dependencia Maven**.

Cada equipo decidirá qué responsabilidades resulta razonable incorporar en este componente común.

El objetivo no consiste simplemente en compartir código, sino en **identificar elementos verdaderamente reutilizables y diseñar una dependencia coherente**.

Durante la defensa deberá poder justificarse tanto aquello que se decidió compartir como aquello que deliberadamente permaneció dentro de cada microservicio.

Persistencia

La solución deberá utilizar al menos un **DBMS relacional** o una **base de datos embebida** para la persistencia de la información.

Como solución base, los diferentes componentes podrán utilizar una instancia común del DBMS.

Esto no implica que todos los microservicios deban compartir necesariamente el mismo modelo de datos ni acceder indistintamente a todas las estructuras existentes.

Esta alternativa **no es limitante**. Será completamente válido utilizar:

- una base de datos por microservicio;
- esquemas diferentes dentro de un mismo DBMS;
- diferentes estrategias de separación de la información;
- otras alternativas que resulten coherentes con la arquitectura propuesta.

El equipo deberá poder explicar qué estrategia adoptó y por qué resulta adecuada para su solución.

APIs REST y documentación

Los microservicios deberán exponer sus funcionalidades mediante **APIs REST**.

Las APIs deberán:

- utilizar apropiadamente los métodos HTTP;
- utilizar códigos de respuesta HTTP coherentes;
- intercambiar representaciones JSON cuando corresponda;
- validar los datos recibidos;
- contemplar y comunicar adecuadamente situaciones de error;
- estar documentadas mediante **OpenAPI / Swagger**.

La documentación deberá representar la API efectivamente implementada.

Contenerización y despliegue

Los componentes necesarios para ejecutar la solución deberán estar contenerizados.

El equipo deberá proporcionar una configuración mediante **Docker Compose** que permita desplegar la solución integral de manera reproducible.

El despliegue deberá contemplar, según corresponda:

- API Gateway;
- microservicios funcionales;
- Keycloak;
- DBMS;
- componentes adicionales desarrollados o incorporados por el equipo;
- configuración necesaria para la comunicación entre ellos.

Las APIs públicas utilizadas por la solución podrán naturalmente permanecer fuera de esta infraestructura.

El objetivo es que la solución pueda ser desplegada como **un sistema completo**, y no como un conjunto de aplicaciones que requieran configuraciones manuales independientes para poder funcionar.

La arquitectura representada en el C2 entregado por el equipo deberá corresponderse con los componentes que efectivamente formen parte de este despliegue.

Nivel de desafío

Los requisitos establecidos en este documento representan el **punto de partida común** para todos los equipos.

No todos los dominios presentan la misma dificultad y no todos los equipos necesariamente elegirán asumir el mismo nivel de desafío.

Se evaluará tanto el cumplimiento de los requisitos mínimos como el **nivel de desafío asumido por el equipo**, considerando especialmente:

- riqueza del dominio seleccionado;
- complejidad y relevancia de las reglas de negocio;
- calidad de la división funcional entre microservicios;
- complejidad de los procesos que atraviesan diferentes componentes;
- relevancia de las integraciones externas;
- decisiones de seguridad;
- decisiones arquitectónicas;
- tratamiento de errores y situaciones excepcionales;
- robustez general de la solución.

Una mayor cantidad de microservicios, tecnologías o integraciones **no implica automáticamente una solución de mayor calidad o complejidad**.

Las extensiones deberán aportar valor al problema planteado y deberán poder ser justificadas por el equipo.

Un equipo podrá decidir profundizar determinados aspectos de la solución y asumir desafíos adicionales siempre que mantenga una arquitectura coherente y una solución funcional.

Proceso de diseño y construcción

El TPI deberá ser entendido como un **proceso iterativo de diseño e implementación**.

No esperamos que la primera idea de arquitectura, la primera división de responsabilidades o la primera solución propuesta sean necesariamente las definitivas.

Esperamos que los equipos:

1. comprendan el problema;
2. construyan una visión compartida;
3. propongan una solución inicial;

4. identifiquen responsabilidades;
5. diseñen la arquitectura;
6. implementen una primera solución;
7. descubran problemas y limitaciones;
8. revisen sus decisiones;
9. evolucionen la implementación;
10. validen integralmente el resultado.

Durante este proceso podrán utilizar las herramientas que consideren convenientes para investigar, diseñar, implementar, probar y documentar la solución.

La utilización de herramientas de asistencia, incluyendo herramientas basadas en Inteligencia Artificial, **no reemplaza la responsabilidad del equipo sobre las decisiones adoptadas ni sobre el código presentado**.

El equipo deberá ser capaz de comprender, modificar y explicar la solución independientemente de las herramientas utilizadas durante su construcción.

Durante la defensa, cualquiera de los integrantes deberá poder explicar las decisiones relevantes y el funcionamiento de la solución construida.

Consideraciones finales

Los dominios propuestos para el TPI son deliberadamente abiertos.

No esperamos que los equipos pregunten qué microservicios deben construir para cada dominio.

Determinar las responsabilidades, establecer los límites entre componentes, identificar las integraciones necesarias y diseñar los procesos de negocio **forma parte del Trabajo Práctico Integrador**.

De la misma manera, no existe una arquitectura única considerada correcta de antemano.

Esperamos soluciones diferentes entre equipos y esperamos que esas diferencias sean consecuencia de las decisiones tomadas durante el análisis y la construcción.

El objetivo final del TPI no será demostrar únicamente que una aplicación funciona, sino demostrar que el equipo es capaz de **comprender un problema y diseñar, construir, integrar, asegurar, desplegar y defender una solución backend completa**.